Stoch - The Quadruped Robot at RBCCPS

Ajay G

Content

- Breif Overview
  - Goal
  - What was achieved by Stoch 1.0 ?
  - How we achieved ?
  - Next what ?
- Stoch 1.0
  - Why we didn't do what google did ?
  - Can it walk with hand-crafted trajectory ?
  - Then what is the point of using DRL ?
- Stoch 2.0
- The Team

Breif Overview

Figure 1: Stoch-1.0 developed at RBCCPS

Goal

To achieve a robust, all terrain, legged robot.

Table 1: Hardware Description of Stoch 1.0.

Hardware	Description
Materials:	Carbon fiber (CF) tubes & 3D printed (PLA)
Servo-motors:	Kondo-2350HV & Robokits Ultra Torque: RKI-1203
IMU:	9 axis sparkfun MPU 9150
Encoders:	Magnetic Encoders, Bourns EMS22a
Power Supply:	Li-Po batteries
	11.1V(3s)×2200mAh for hip motors
	7.4V(2s)×3000mAh for knee motors
PWM multiplexer:	Adafruit 16-Channel 12-bit: PCA968
Mass:	3.0 Kg
Total Leg Length	230 mm

What was achieved by Stoch 1.0?

The quadruped performed a variety of gaits such as walks, bounds, trots, etc. . . as show in this video

How we achieved?

We used on-policy, model-free DRL algorithm based on actor-critic learning framework called Proximal Policy Optimisation (PPO) to generate the gaits in the simulation(PyBullet). It took 30 Million samples (5-7 hours) to learn a particular gait in the simulation. To transfer it on the hardware, we took the end-effector trajectory form simulation and did the principal component analysis to generate different gaits and we deployed on the hardware. You can find more info in this paper (my name is in the acknowledgment).

Next what?

1. How to learn different tasks with least samples?

   Model-based RL, & Imitation Learning.

2. How to control & combine different behavior such as turning, climbing stairs, reacting to external disturbance?

   Funnel Libraries, & Reactive Control.

I would like to mention here more on the practical issues(behind the paper stories) and evolution of this project, especially on stoch 2.0

Stoch 1.0

The project was initially motivated by Googles' work on Sim-to-Real: Learning Agile Locomotion For Quadruped Robots. The essence of the paper is to improve the transferability from learned control policy from simulation to hardware by:

1. Improving the Robot Model by having:
    1. Accurate URDF by physically measuring many parameters
    2. Actuator Model
    3. Latency (communication Delay)
2. Adding Noise to have Robust Controller:
    1. Randomising Dynamic Parameters such as Mass, Friction, etc...
    2. Random Perturbation
    3. Using a compact observation space
3. Control of the learned policies

    Instead of learning the entire policy from scratch which will result in a lack of controllability, they decompose the problem into two parts:

    1. $\bar{a}(t)$ An open loop component that allows a user to provide reference trajectories
    2. $\pi(o)$ feedback component that adjusts the leg poses on top of the reference based on the observations.

$$a(t, o) = \bar{a}(t) + \pi(o)$$

We can infer two important things:

1. We need to have a very good model to produce good deterministic baseline motion which are vital for safety using classical control.
2. Fine tuning of the motion is more likely to be better handled by learning based control.

Why we didn't do what Google did?

Well, getting accurate model is vital, it is not a trivial task, for example, google disassembled their robot and weighed each component then, modeled their URDF. Our robot roughly cost around 1,200 USD was made out of hobby grade stuff and was very fragile to disassemble. Plus we didn't have accurate torque sensor to model the actuator. For a perspective, MIT Cheetah uses ATI force sensor which is roughly the cost of our entire robot. Google had two controllers, one high-level controller TX2 and another low-level motor driver. We had three Ubuntu to RPI3 (via MQTT Ethernet) and from RPI3 to Servo (via PWM multiplexer) which increased our latency and unpredictableness. All these factors

made us look for an alternate approach. So we took the end-effector trajectory and used hand-crafted IK solver in Python to run it on our hardware.

Can it walk with hand-crafted trajectory ?

It does. If you hand-craft a trajectory, let us say an ellipse or circle, it will still walk. and adjust the phase difference to get different gaits.

Then what is the point of using DRL ?

Well, the handcrafted may or may not be optimal, but the generated gaits were certainly closer to optimal, given their similarity to other quadrupeds.

Stoch 2.0

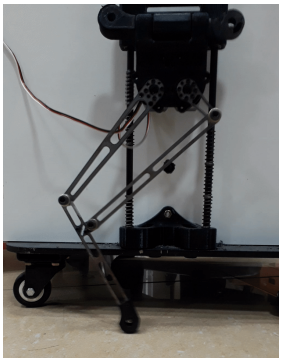

Figure 2: Stoch 2.0 Leg-Testing

As mentioned earlier stoch 1.0 was too fragile. It had a lot of issues, so we wanted to have a robust hardware, "Stoch 2.0". Soon more details would be updated on Stoch 2.0.

The Team

Team Members: Shounak Bhattachary, Dhaivat Dholakiya, Abhik Singla and myself..!

Principal Investigators: Shishir N. Y. Kolathaya, Prof. Bharadwaj Amrutur

I would like to specially thank **Shishir**, & **Prof. Bharadwaj**, for giving me a wonderful oppourtinity to work in this project.

Figure 3: Robotics Group at RBCCPS

Figure 4: Stoch Team